FitHub - Intelligent Smart Gym Ecosystem

Phase 3 abstract (making of)
Lokesh Chandra Chowdary Battula | 10252426
Project: Software Engineering | CSEMCSPSE01

Solution and customer value

FitHub addresses fragmented gym visits for Members who otherwise manage membership, attendance, training records, classes and availability through separate processes. Trainers support assigned Members, while Administrators control plans, access, safety and facilities. The responsive application connects registration, consent, onboarding, a EUR 29.90 Monthly membership, QR attendance, workouts, progress, guidance, classes, notifications, equipment, faults and parking.

The value is reduced uncertainty: membership and payment state are visible, sessions are recorded, guidance explains its source data, and equipment or parking changes appear without refresh. Role boundaries keep personal Member services separate from Trainer and Administrator operations.

Method and scope control

The Waterfall sequence established requirements and architecture before staged implementation, Phase 2 manual testing, Phase 3 corrections, deployment and final acceptance. MoSCoW prioritisation protected complete Must Have workflows. Synthetic accounts made defects reproducible without real personal data; incomplete or unmeasured work remained explicit technical debt.

Architecture and technical concept

Angular 22 and TypeScript provide the browser interface. ASP.NET Core 10 separates Domain rules, Application contracts, Infrastructure adapters and the API. EF Core persists authoritative PostgreSQL state. Identity, memory-only access tokens, rotating HttpOnly refresh cookies and policies protect role and ownership boundaries. SignalR publishes committed changes; Stripe Test Mode handles Checkout and signed webhooks; Mailpit and Brevo deliver local and hosted password-reset mail.

Connected virtual laboratories

The virtual laboratories test smart-system behaviour without claiming physical ESP32, NFC, ultrasonic or camera delivery. Connected Mode submits synthetic card, presence and parking observations through Administrator-only endpoints. Normal services validate them, PostgreSQL stores accepted transitions, and the standard SignalR hubs update simulator and operational pages from one authoritative backend state.

Implementation and integration

Implementation progressed through authentication and profile; membership, payment and attendance; workouts, progress and dashboard; equipment and faults; parking and recommendations; then administration, classes and notifications. EF migrations preserved history while adding consent evidence, rest seconds, reset tokens and notification preferences. Duration and rest remain separate without changing volume or record calculations.

Membership activation never trusts a redirect. The API prices Checkout from PostgreSQL, verifies Stripe's signed Test Mode event or server-side reconciliation, and activates one payment idempotently. Attendance tokens are single-use, reset tokens are random and hash-only, and preferences suppress optional notifications before creation.

Testing and corrected defects

Final verification combined regression suites with nine hosted manual cases. The release passed 119 backend unit, 95 PostgreSQL integration and 142 Angular tests, plus both simulator suites. All nine acceptance cases passed. Exact synthetic Members, amounts, QR events, sets, machine states, parking spaces and class capacities replaced vague valid-data descriptions.

Corrections covered form submission, workout rest, consent, secure reset, preferences and the 30-second shared-equipment clearance sequence. Hosted testing exposed PostgreSQL retry-boundary defects in payment, booking and reset, plus fault-comment, full-class and Member SignalR errors. Original failures remained recorded and were retested.

Public deployment

The public release uses Render for Angular and the API, Neon PostgreSQL, Brevo and Stripe Test Mode. This replaced the initially considered Azure plan while retaining HTTPS, WebSockets, persistent data and email within free-tier constraints. Docker Compose reproduces PostgreSQL, API, frontend and Mailpit locally; GitHub Actions verifies candidates.

Lessons learned and remaining work

A working screen is not proof of integration; redirects cannot replace verified payment confirmation; retry strategies must wrap whole transactions; and real-time features require one backend authority. Concrete manual data exposed defects that isolated tests missed, while virtual laboratories made state transitions repeatable. MoSCoW protected complete core workflows, and hosted behaviour required separate verification from localhost.

Remaining debt includes incomplete mobile coverage, no approved 20-user p95 result, ephemeral optional photos, a redundant Stripe Test webhook and free-tier cold starts. Physical hardware, broader accessibility, persistent object storage and load testing remain future work.